

Files

FUNDAMENTALS OF PROGRAMMING

Bùi Duy Đăng

bddang@fit.hcmus.edu.vn

Plan for today

- Stream
- Motivation
- Files

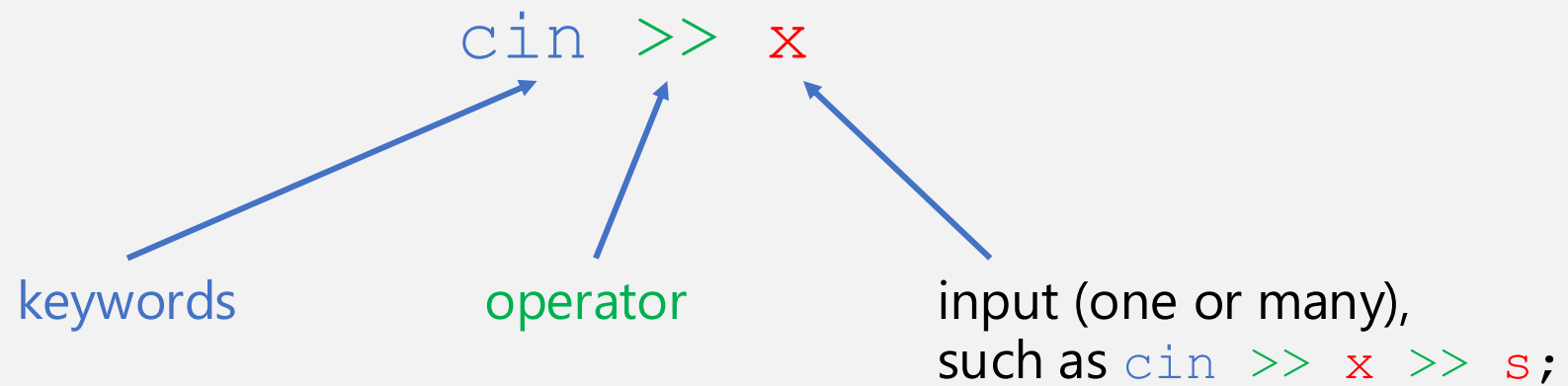
Example

```
int x;  
cin >> x;
```

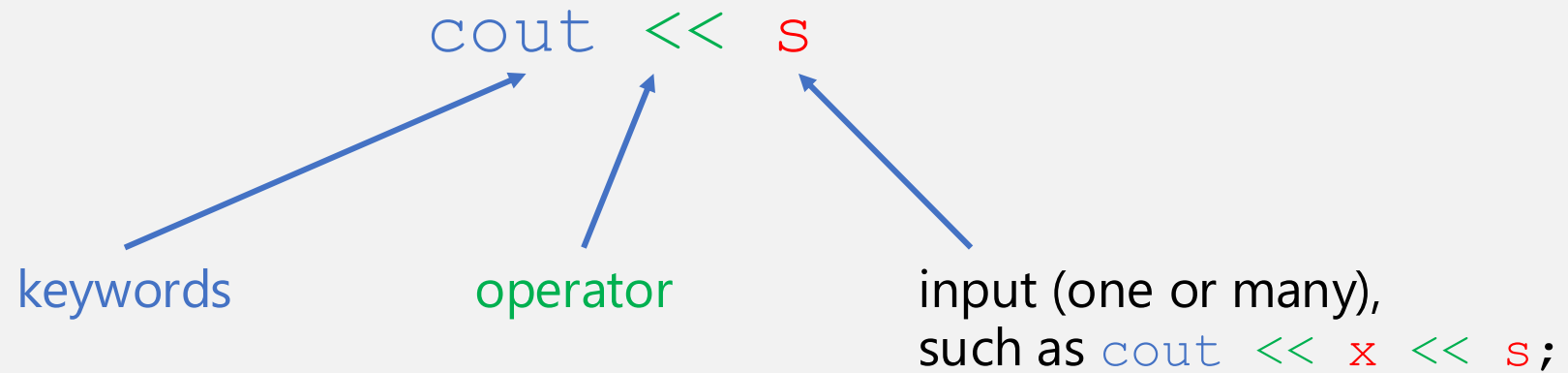
```
char s[100];  
cin.getline(s, 100);
```

```
cout << x << endl;  
cout << s << endl;  
// how they work?
```

Example



Example



How it works

- C++ stores data (variables, arrays, structs, etc.) **in RAM**
- Data are fetched into RAM and used in programs via **device**
 - Input devices: keyboards, mice
 - Output devices: screens, printers
 - Input/Output devices: files
- Devices are processed via **stream**

Stream

- A C++ **stream** is a flow of data **into** or **out** of a program, such as the data written to `cout` or read from `cin`.
- Input stream
 - Using `>>` operator (**extraction** operator) to get input data
 - Such as, `istream`
- Output stream
 - Using `<<` operator (**insertion** operator) to output data
 - Such as, `ostream`

Stream

- **istream** is a general purpose input stream. `cin` is an example of an `istream`.
- **ostream** is a general purpose output stream. `cout` and `cerr` are both examples of `ostreams`.
- **ifstream** is an input file stream. It is a special kind of an `istream` that reads in data from a data file.
- **ofstream** is an output file stream. It is a special kind of `ostream` that writes data out to a data file.

Stream

- Text stream
 - Texts only
 - Ended by '\0' or '\n'
- Binary stream
 - Bytes only
 - It works specifically with bytes when reading and writing
 - Can handle any kinds of data, even texts

Files

Motivation

- We want to store information of students in a class

Input: from a keyboard

Output: to a screen

Store: in RAM

→ Waste time and when a program is terminated, data in RAM will be released.

- Need a way to store data that can be **used when a program is terminated**, can be **used many times**, and **whose size are not limited**

→ Using **files**

Files

- A file is a container in a computer system for **storing information**.
- Files can be stored on **optical drives**, hard drives or other types of storage devices.
 - Still alive when a program is terminated
 - Size is not limited (depend on the size of devices)
- Allow to read (input device) and write (output device) data

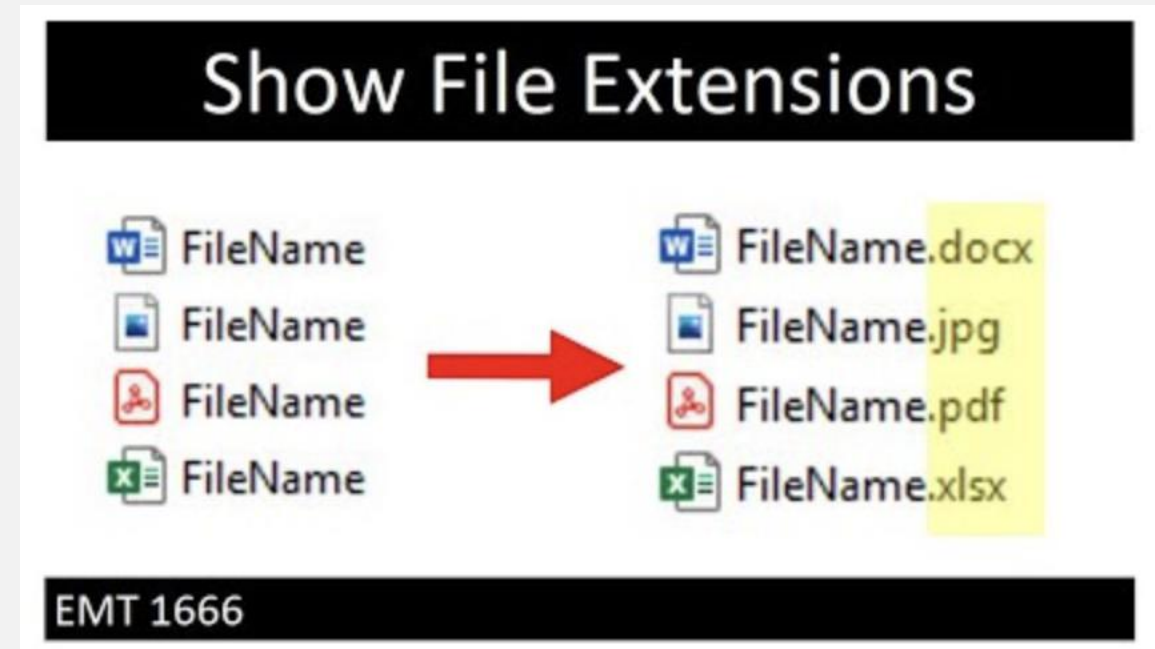
Categorization

- For end-user perspective: only care the content based on types of files, such as `.exe`, `.doc`, `.ppt`, etc.
- For programmer perspective: use `stream` to process with files
 - Text files (text stream)
 - Binary files (binary stream)

File name

name.extension

- **name:**
 - must have,
 - length is depended on Operating System
- **extension**
 - not required,
 - often contains 3 characters



[Credit: ExcellsFun](#)

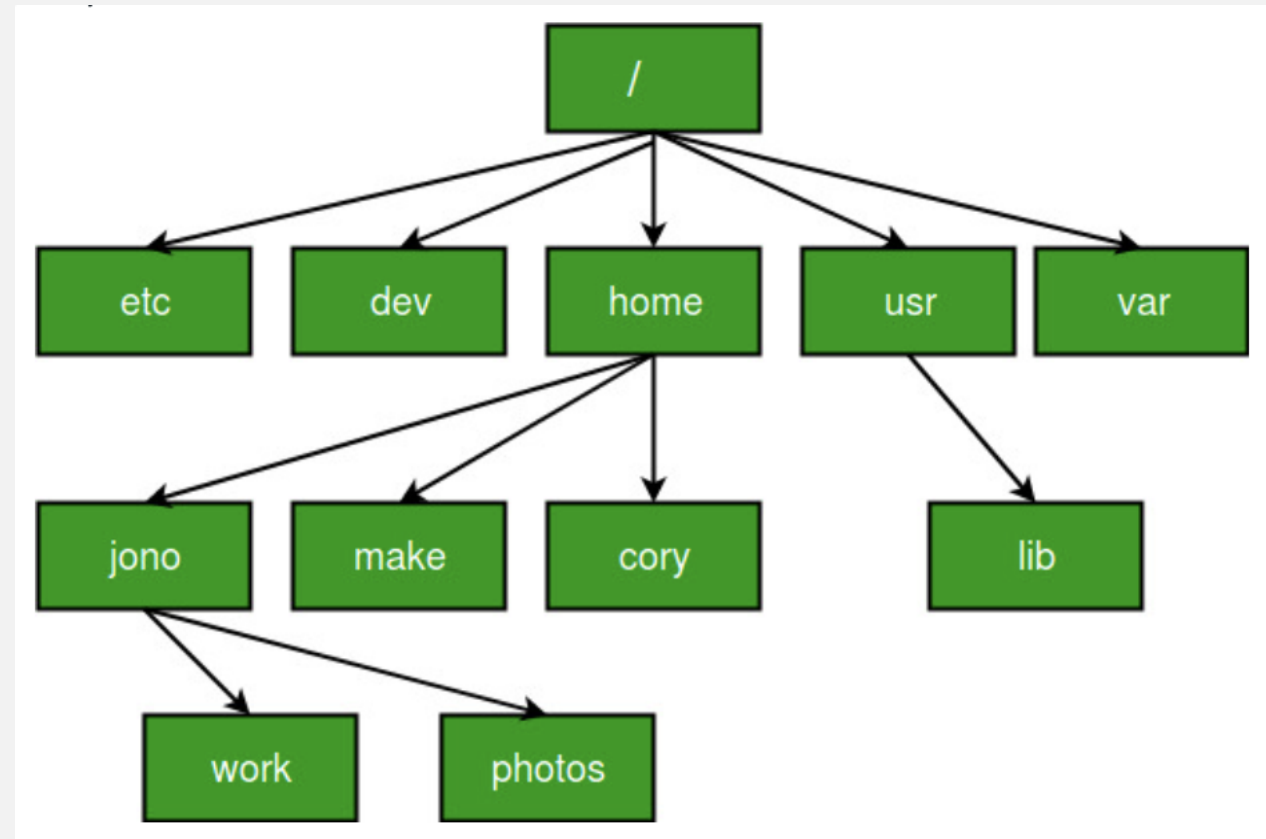
File path

- Absolute path
 - An absolute path is defined as the specifying the location of a file or directory **from the root directory** (/ or \)
- Relative path
 - Relative path is defined as the path related to the **present working directly** (pwd). It starts at your **current directory** and never starts with a / (or \)

Note: in a program, use "\\\" for displaying '\\'. For example:
"C:\\StudyingMaterials"

File path

- Absolute path
E.g., `/home/make/`
- Relative path
E.g., `./home/make`
`./make`



[Credit: geeksforgeeks](#)

A flow when handling with files

- **Open a file:**
create a file stream to connect with a desired file
- **Process data in a file** (after opening the file)
 - **Read** data in the file to a program
 - **Write** data from a program to the file
- **Close a file** (after using the file)

File library

- There are three classes included in the `fstream` library, which are used to create, write or read files:

`ofstream`: to create/write on files

`ifstream`: to read from files

`fstream`: to both read and write from/to files

Open a text file

- In C++, we can open a file using the `ofstream` and `ifstream` classes

```
#include <fstream>
```

```
...
```

```
ofstream fOut1("example.txt");
```

```
ofstream fOut2; fOut2.open("example.txt");
```

```
// it is similar to the codes below
```

```
fstream fOut;
```

```
fOut.open("FileName.txt", ios::out);
```

Open a text file

- In C++, we can open a file using the `ofstream` and `ifstream` classes

```
#include <fstream>
```

```
...
```

```
ifstream fIn1("example.txt");
```

```
ifstream fIn2; fIn2.open("example.txt");
```

```
// it is similar to the codes below
```

```
fstream fIn;
```

```
fIn.open("FileName.txt", ios::in);
```

Open a text file

- Syntax

```
open (FileName, Mode) ;
```

`FileName` is a string representing the name of the file

`Mode` is an optional parameter with a combination of the following flags

Open a text file

- Syntax

```
open (FileName, Mode) ;
```

Mode is an optional parameter with a combination of the following flags

<code>ios::in</code>	Open for input operations.
<code>ios::out</code>	Open for output operations.
<code>ios::binary</code>	Open in binary mode.
<code>ios::ate</code>	Set the initial position at the end of the file. If this flag is not set, the initial position is the beginning of the file.
<code>ios::app</code>	All output operations are performed at the end of the file, appending the content to the current content of the file.
<code>ios::trunc</code>	If the file is opened for output operations and it already existed, its previous content is deleted and replaced by the new one.

Example

```
...  
fstream file;  
file.open("FileName.txt", ios::out);  
if (!file)  
    cout<< "Error while creating the file";  
else{  
    cout << "File created successfully";  
    file.close();  
}
```

Writing to text file

- using `std::ofstream`

Open file for writing: `open`

Close file after writing: `close`

Take input (same as `cout`, insertion operator): `<<`

```
ofstream file;  
file.open("FileName.txt"); //  
file << "hello world!" << endl; // how it works?  
file.close();
```

Reading from text file

- `using std::ifstream`
 - Open file for reading: `open`
 - Close file after reading: `close`
 - Take input (same as `cin`, extractor operator): `>>`

```
ofstream file;  
int n;  
file.open("FileName.txt"); //  
file >> n;    // how it works?  
file.close();
```

Example

```
// ... declare libraries
int main() {
    fstream fIn("example.txt", ios::in);
    // ... check file opens?
    int a[100], n;
    fIn >> n; // n = ?
    for (int i = 0; i < n; i++) fIn >> a[i];
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    fIn.close(); return 0;
}
```

Example.txt

5

1 2 3 4 5

Some useful fuctions

- `eof()` ; // returns True if the end of the file (EOF) is reached, with False returned otherwise.
- `is_open()` ; // returns True if a stream object is open.
- `fail()` ; // returns True if an error occurs, but we can still operate on the stream.
- `good()` ; // returns True if each of the other three states are false, i.e., the stream is neither bad nor failed, nor has it reached the end of the file.
- `.get()` ; // get one character from a file stream named FileStream.

single character (1) `int get();istream& get (char& c);`

c-string (2) `istream& get (char* s, streamsize n);istream& get (char* s, streamsize n, char delim);`

stream buffer (3) `istream& get (streambuf& sb);istream& get (streambuf& sb, char delim);`

Other topics

- Stream buffer
- Position indicator

Exercises

- 1) Write a program to get 3 integers from keyboards to a file
- 2) Write a program to read 3 integers named a, b, c from a file, then solve an 2nd degree equation $ax^2+bx+c=0$ and write the result to another file
- 3) Write a program read n intergers from a file, then sort them in an ascending way

E.g.,

4	→	4
2 5 1 4	→	1 2 4 5

Exercises

- 4) Write a program to record messages inputted from a keyboard to a file.
- 5) Write a program to show full contents of a file to a screen.
- 6) Write a program to count numbers of alphabet characters of a file and store it to another file.
- 7) Write a program to count words of a file and store the result to another file

Exercises

8) Write a program to count duplicated words in a file and store the result to another file.

E.g., InputFile → Hello world, my name is Hello.

OutputFile → Hello: 2 | world: 1 | my: 1 | name: 1 | is: 1

9) Write a program to concatenate two text files into one.

10) Write a program to copy a file to another file.

11) Write a program to store a list of student information to a file and load them to a screen.